Data Estate Insights & Data Quality — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Data Estate Insights & Data Quality (Microsoft Purview) 5

What you get . . . 5

What this delivers . . . 5

The deployment model . . . 6

Outcome . . . 6

1. Solution Overview — Data Estate Insights & Data Quality (Microsoft Purview) 6

What this solution is . . . 6

Why this matters . . . 6

The two building blocks . . . 6

Governance domains, data products, and roles . . . 7

Data-quality scanning reads live data — safe by design . . . 7

What a "pilot" means here . . . 7

Licensing and the evolving portal . . . 7

What this is not . . . 8

The deployment model . . . 8

2. Requirements Guide — Data Estate Insights & Data Quality (Microsoft Purview) 8

2.1 A governed estate already mapped, scanned, and catalogued . . . 8

2.2 Least-privilege insights + data-quality roles . . . 9

2.3 Data Quality enabled and its compute available (verify licensing/preview) . . . 9

2.4 A pilot data product with critical data elements identified . . . 9

2.5 A source connection and credentials for the data-quality scan (never in code) . . . 9

2.6 Agreed quality dimensions, rules, and thresholds (owner-set, not technical defaults) . . . 10

2.7 A data-owner / steward review path and privacy sign-off . . . 10

2.8 A pilot scope and change control . . . 10

Checklist . . . 10

Global Requirements — Microsoft Purview Solutions 11

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11

2. Purview roles (least-privilege) and portal access . . . 11

3. Security & Compliance PowerShell + Graph toolchain . . . 12

4. Data sources & scope in scope . . . 12

5. Compliance, records, privacy & legal stakeholders . . . 12

6. A pilot scope and change control . . . 13

3. Implementation Guide — Data Estate Insights & Data Quality (Microsoft Purview) 13

Step 1 — Assess current state (read-only) . . . 13

Step 2 — Design the insights and data-quality scope . . . 13

Step 3 — Record the pilot scope (read-only manifest) . . . 14
Step 4 — Stand up Data Estate Insights and take a baseline (read-only) . . . 14
Step 5 — Establish the data-quality connection and credentials (safe by design) . . . 14
Step 6 — Author data-quality rules on the pilot data product . . . 14
Step 7 — Pilot the data-quality scan on the bounded scope (test-first) . . . 15
Step 8 — Review the scores with the owner and tune rules . . . 15
Step 9 — Wire estate-health tracking (health controls, if available) . . . 15
Step 10 — Validate against the pilot gate . . . 15
Step 11 — Expand across the estate (enforce) . . . 16
5. Testing Guide — Data Estate Insights & Data Quality (Microsoft Purview) 16
Test 1 — Estate and roles are ready . . . 16
Test 2 — Data Estate Insights baseline is captured . . . 17
Test 3 — The data-quality connection is reachable . . . 17
Test 4 — Data-quality rules are authored on the pilot data product . . . 17
Test 5 — The data-quality scan succeeds and returns scores . . . 17
Test 6 — Scores are meaningful and owner-reviewed . . . 17
Test 7 — Estate-health coverage is reported . . . 17
Test 8 — Trend tracking works against the baseline . . . 17
Pass/fail gate . . . 18
6. Troubleshooting Guide — Data Estate Insights & Data Quality (Microsoft Purview) 18
A. Cannot reach the account / token fails . . . 18
B. Data Quality surface does not respond (feature availability) . . . 18
C. Insights show no data . . . 19
D. The data-quality connection or Test connection fails . . . 19
E. The data-quality scan fails or returns no scores . . . 20
F. Scores look wrong . . . 20
G. Rolling back a rule or scan . . . 20
H. Health / coverage gaps . . . 21
I. Diagnostics quick reference . . . 21
J. When to escalate . . . 21
7. Best Practices — Data Estate Insights & Data Quality (Microsoft Purview) 21
8. Deliverables Summary — Data Estate Insights & Data Quality (Microsoft Purview) 22
Documentation . . . 22
Named artifacts . . . 23
PowerShell scripts . . . 23
What the buyer receives . . . 24

Outcome 24
Appendix A — PowerShell Scripts 25

Executive Summary — Data Estate Insights & Data Quality (Microsoft Purview)

A data estate can be fully mapped and classified and still be full of data nobody trusts — customer records with missing emails, finance extracts with inconsistent formats, account IDs with duplicates, a "gold" data product quietly gone stale. Leaders have no single view of estate health and no quality score to point at. That is a **compliance gap** — you cannot evidence the health of the data you govern — and a **data-risk exposure**, because reports, decisions, and AI are built on data of unknown quality. This solution closes it: it stands up **Data Estate Insights** (the reporting layer that shows estate health) and **Data Quality** in the Unified Catalog (data-quality **rules and scoring** across the governed estate's critical data) — configured, piloted on a bounded data product, and validated.

This is a complete, do-it-today package written for an engineer standing up insights and data quality on an already-governed Purview estate. Every artifact is named once and reused everywhere, and data-quality scoring — which reads live data and consumes compute — is proven on a bounded pilot before the estate is scored. Because the insights and Data Quality experiences are evolving, consumption-/ plan-gated, and partly in preview, the guide cites the method and flags every capability "verify your licensing/tenant" rather than asserting a path, an endpoint, or a price as permanent.

What you get

• A plain-language Solution Overview of the two building blocks — Data Estate Insights (asset,

classification, and ownership coverage; scan and catalog health) and Data Quality (critical data elements, rules, and scoring across dimensions such as completeness, conformity, and uniqueness) — and how they complement the Data Map/catalog foundation.

• A Requirements Guide with verify / implement-if-missing for every prerequisite: a governed estate,

least-privilege insights and data-quality roles, Data Quality enabled and its compute available, a pilot data product with critical data elements, a source connection via managed identity / Key Vault, agreed dimensions and thresholds, and a data-owner review path with privacy sign-off.

• An Implementation Guide of precise, ordered steps: design the scope, baseline insights read-only,

author data-quality rules on a pilot data product, **pilot the data-quality scan** on that bounded scope, review and tune the scores with the owner, wire estate-health tracking, validate against the gate, then expand with scheduled scoring.

• Five PowerShell tools for readiness discovery, pilot-scope capture, rule authoring and scoring

(`-WhatIf` by default), an insights + data-quality scoring report with a baseline, and a scope/health helper — authentic Purview data-plane REST automation with **no credentials in code**.

• Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

A measured data estate: health reported through Data Estate Insights, and data quality scored per rule, per data asset, and per data product across the governed estate's critical data — proven on a pilot before you scale.

The deployment model

Assess → Design → Configure → **Pilot (bounded data-quality scan)** → Validate → Expand — with a pilot gate before any estate-wide or scheduled scoring.

Outcome

The estate's health and trustworthiness made visible, scored, and defensible — with the business (not IT alone) able to see coverage and quality — and the threshold, remediation, and compliance determinations left to the data owners, stewards, and compliance functions who make them.

1. Solution Overview — Data Estate Insights & Data Quality (Microsoft Purview)

What this solution is

Once a data estate is mapped, scanned, and catalogued in Microsoft Purview, two questions remain: *"how healthy is the estate as a whole?"* and *"can I trust the data inside it?"* This solution delivers both. It stands up **Data Estate Insights** — the reporting and analytics layer that shows the health of the governed estate (asset coverage, classification distribution, ownership coverage, scan and glossary health) — and **Data Quality** in the Unified Catalog: **data-quality rules and scoring** across the governed estate's critical data, so a data product carries a measurable quality score, not just a classification.

> ***Boundary.*** *This solution provides the technical method to *measure and report* estate health and data quality — configure insights/analytics, define data-quality rules, run scoring on a bounded scope, and read the results. It does* ***not*** *make the business or regulatory determinations: what a "critical data element" is, which quality threshold is acceptable, whether a low score is a compliance problem, or what remediation the business must take — those belong to the organization's* ***data owners, stewards, compliance, privacy, and legal*** *functions. The solution gives them a measured, evidenced estate to make those decisions against.*

Why this matters

A mapped and classified estate can still be full of data nobody trusts. A customer table has 12% missing emails, a finance extract carries dates in three formats, an account-ID column has duplicates, and a "gold" data product is quietly stale. Leaders cannot see this: they have no single view of estate health and no quality score to point at. The result is a **compliance gap** (you cannot evidence the health of the data you govern) and a **data-risk exposure** (decisions, reports, and AI are built on data of unknown quality). This solution makes the estate's health and quality **visible, scored, and defensible** — configured, piloted on a bounded scope, and validated.

The two building blocks

This solution stands up two capabilities on top of an already-governed estate (a Purview account with sources mapped, scanned, and curated into the Unified Catalog — see the *Data Governance: Data Map, Scanning & Unified Catalog* solution if that foundation is not yet in place).

1. **Data Estate Insights** — the analytics/reporting layer over the governed estate. It aggregates

what scanning and cataloguing produced into an estate-level picture: how many assets are known and scanned, the **classification distribution** (where the regulated data is), **ownership coverage** (assets with a named owner vs unowned), scan health, and glossary/data-product coverage. In the evolving Purview portal this appears as **reports / analytics / Health management** over the Unified Catalog; classic Purview called it **Data Estate Insights**. Names and paths are evolving — **verify current availability and portal paths for your tenant**.

1. **Data Quality** — the Unified Catalog capability that measures whether the data itself is fit for

use. You bind **critical data elements** (the columns that matter) inside a **data product** to **data-quality rules**, run a **data-quality scan** that profiles and evaluates the data, and get a **quality score** per rule, per data asset, per data product, and rolled up per governance domain. Rules are grouped by **quality dimensions** Microsoft documents — for example **Completeness, Conformity, Consistency, Accuracy, Uniqueness/Validity, and Timeliness**. Data Quality is licensing-/consumption-dependent and parts may be in **preview** — **verify your licensing/tenant**.

Governance domains, data products, and roles

Both capabilities are organized by the Unified Catalog's structure: **governance domains** (business groupings), **data products** (curated, published sets of assets with an owner and a use purpose), and the **critical data elements** inside them. Access uses least-privilege governance roles — for example a **Data Governance / catalog reader** to view insights, a **governance domain owner** and **data steward** to curate, and a **data-quality steward** to author and run data-quality rules. Global or account-level roles are used only where account-level changes are required. Exact role names are evolving — **verify current role names for your tenant**; the Implementation Guide names the least-privilege role at each step.

Data-quality scanning reads live data — safe by design

A data-quality scan **profiles and reads the actual data** in the bound source to evaluate the rules (unlike a classification scan, which samples for pattern-matching). That has two consequences this solution treats as first-class: **privacy** (the scan reads real values, so it runs least-privilege, on a bounded scope, with the data owner's agreement) and **credentials** (the scan authenticates to the live source via the Purview account's **managed identity** or an **Azure Key Vault**–stored secret — **never a secret in a script or document**). Rules are always **validated on a bounded pilot data product first**, and the score is reviewed with the owner before the estate is scored at scale.

What a "pilot" means here

Data Quality does not *enforce* anything on the data — it **measures** it. So piloting means running rules and scoring on **one bounded data product first**, in a **test/limited** scope: validate that each rule matches the owner's intent, that the score is meaningful, and that the scan cost and runtime are acceptable — *before* pointing data-quality scoring at the whole estate or scheduling it. Estate Insights is read-only reporting, so it is safe to view at any time; the pilot discipline applies to **data-quality rule authoring and scanning**, which consume compute and read live data.

Licensing and the evolving portal

Data Estate Insights / analytics and Unified Catalog **Data Quality** are **licensing-/plan- and consumption-dependent** (advanced Purview capabilities typically require **Microsoft 365 E5 / E5 Compliance** and/or **consumption billing on an Azure subscription**), and some features may be in **preview**. The portal experience, feature names (Insights vs Analytics vs Health management), and billing are **actively evolving**. This solution **cites the method and flags every capability "verify your licensing/tenant"** — it never asserts a portal path, an API version, or a price as permanent.

What this is not

This is the *measure-and-report* solution for a governed estate. It is **not** the foundational Data Map / scanning / cataloguing solution (that builds the estate this one measures), and it is **not** a records, retention, DLP, or insider-risk configuration. It complements them: the estate is mapped and classified elsewhere; this solution tells you **how healthy and how trustworthy** that estate is.

The deployment model

> *Assess the governed estate and readiness, design the insights and data-quality scope (domain, data product, critical data elements, rules), configure insights and author rules, **pilot data-quality scoring on a bounded data product**, validate the scores and the health picture against the gate, then expand across the estate — scheduling scoring only after the pilot passes. The repeatable way: **Assess** → **Design** → **Configure** → **Pilot (test)** → **Validate** → **Expand.***

2. Requirements Guide — Data Estate Insights & Data Quality (Microsoft Purview)

Verify every requirement before you configure insights or author a data-quality rule. Each entry gives Description, Why, How to verify, and How to implement if missing. Category-wide prerequisites (tenant, compliance licensing, least-privilege roles, toolchain, in-scope data, stakeholders, pilot) are in `global-requirements-purview.md` — this guide adds the prerequisites specific to **Data Estate Insights and Data Quality**.

> ***Licensing/availability caveat.** Data Estate Insights / analytics and Unified Catalog **Data Quality** are licensing-/plan- and **consumption**-dependent, and some features may be in **preview**. Where this guide names a capability, portal path, or role, **verify its current availability, naming, and pricing for your tenant** in Microsoft's documentation and the Purview portal before relying on it — the experience is evolving.*

2.1 A governed estate already mapped, scanned, and catalogued

Description: A Microsoft Purview account whose sources are already **registered, scanned, and classified**, with a working **Unified Catalog** (at least one **governance domain** and one **data product**). This solution measures and reports on that estate; it does not create it. Why: Insights aggregate scanned/catalogued assets, and data-quality rules bind to **data assets inside a data product** — without a governed estate there is nothing to report on or score. How to verify: The Purview account endpoint (`https://<account>.purview.azure.com`) resolves and the catalog returns assets; run `DGINSIGHT-Discovery.ps1` to confirm assets, domains, and data products exist. How to implement if missing: Stand up the Data Map / scanning / Unified Catalog foundation first (see the *Data Governance:

Data Map, Scanning & Unified Catalog* solution), then return here.

2.2 Least-privilege insights + data-quality roles

Description: Role assignments at the right scope — a **catalog / Data Governance reader** to view insights, a **governance domain owner** and **data steward** to curate, and a **data-quality steward** to author and run data-quality rules. Account-level or **Data Governance Administrator** only where account-level changes are required. **Verify current role names for your tenant** — Unified Catalog roles are evolving. Why: Purview uses fine-grained governance roles; viewing insights, authoring rules, and running scoring each require the matching role, and least-privilege is a governance requirement in its own right. How to verify: In the Purview portal open the governance domain's **roles** and the data-quality role page; run `DGINSIGHT-Discovery.ps1` (reports role/feature coverage, read-only). How to implement if missing: Assign the least-privilege role at the narrowest domain scope under change control; reserve Data Governance Administrator for a small named group.

2.3 Data Quality enabled and its compute available (verify licensing/preview)

Description: The Unified Catalog **Data Quality** capability enabled for the tenant, with the **compute** it uses to profile and evaluate data available (Data Quality runs a scan that reads source data). Confirm the capability is licensed/enabled and not blocked by a preview gate. Why: Rules cannot run and scores cannot be produced without Data Quality enabled and its scan compute available; this is the most common "the feature isn't there" blocker and it is plan-/preview-gated. How to verify: Data Quality is visible in the Unified Catalog for a data product and a test rule can be created; `DGINSIGHT-Discovery.ps1` reports whether the data-quality surface responds. How to implement if missing: Confirm the required licensing/consumption model and enablement with your Purview/Azure owner; if the feature is in preview or unavailable in your tenant/region, record it and defer data-quality scoring — **do not assume availability**.

2.4 A pilot data product with critical data elements identified

Description: One bounded **data product** chosen as the pilot, with its **critical data elements** (CDEs) — the specific columns that matter (for example `CustomerID`, `Email`, `AccountBalance`) — identified with the data owner, and the **data assets** that back them mapped into the data product. Why: Data-quality rules bind to critical data elements inside a data product; without an agreed CDE set there is nothing to write rules against. How to verify: The pilot data product exists in the catalog with its assets mapped and 3–10 CDEs named; run `DGINSIGHT-Set-Scope.ps1` to record the pilot scope manifest. How to implement if missing: Facilitate a short workshop with the data owner to pick one high-value data product, confirm its backing assets, and name the CDEs to measure.

2.5 A source connection and credentials for the data-quality scan (never in code)

Description: A working **connection** from Data Quality to the pilot data product's backing source, with credentials via the Purview account's **managed identity** or an **Azure Key Vault**–stored secret. The scan **reads live data**, so the identity is granted the **least read** role on the source only. **No credential is ever placed in a script or document.** Why: A data-quality scan profiles and evaluates real values; it must authenticate to the live source, and storing that credential safely (managed identity / Key Vault,

least-privilege) is mandatory and is what keeps the provided scripts secret-free. How to verify: The data-quality **connection / Test connection** for the pilot source succeeds and the identity has only read; no secret appears in any script. How to implement if missing: Configure the connection, grant the managed identity (or a Key Vault–stored secret) the least read role on the source, and confirm the test succeeds before scoring.

2.6 Agreed quality dimensions, rules, and thresholds (owner-set, not technical defaults)

Description: With the data owner, agree which **quality dimensions** to measure for each CDE — for example **Completeness, Conformity, Consistency, Accuracy, Uniqueness/Validity, Timeliness** — the specific **rule** per dimension (e.g. "Email matches format", "CustomerID is unique"), and the **threshold** that counts as pass/fail (e.g. completeness ≥ 98%). Thresholds are a **business determination**, not a technical default. Why: A score is only meaningful against an agreed threshold; the dimensions and thresholds are the owner's call, and the solution measures against them rather than inventing them. How to verify: For each pilot CDE the dimension(s), rule, and threshold are written down and confirmed by the owner; `DGINSIGHT-Set-Scope.ps1` records them. How to implement if missing: Run the CDE workshop (2.4) far enough to capture, per CDE, the dimension, rule intent, and pass threshold before authoring rules.

2.7 A data-owner / steward review path and privacy sign-off

Description: A named **data owner / steward** to review the scores and decide what a low score means, plus a **privacy acknowledgement** that a data-quality scan reads live data on the bounded pilot scope. Why: The solution produces scores; the owner interprets them and owns remediation, and because scoring reads real data, a privacy-aware, least-privilege, bounded pilot is required. How to verify: The owner/steward is named and has agreed the pilot scan reads live data on the bounded scope; determinations (thresholds, remediation, compliance meaning) are assigned to them. How to implement if missing: Identify the owner/steward and capture their agreement to the bounded pilot before running any data-quality scan.

2.8 A pilot scope and change control

Description: A **bounded pilot** — one governance domain, one data product, its CDEs and rules — to prove insights and data-quality scoring before estate-wide rollout, plus a **change-control** window for scans (data-quality scans consume compute and read live data). Why: Every solution in this category follows Assess → Design → Configure → **Pilot** → Validate → Expand; data-quality scanning is compute- and data-access-sensitive and is always proven on a bounded scope first. How to verify: A named pilot domain/data product, its CDE/rule set, and a scan change window exist; run `DGINSIGHT-Set-Scope.ps1` to materialize the pilot scope. How to implement if missing: Choose one well-understood, high-value data product as the pilot; agree the scan window and change control with the data owner.

Checklist

- Governed estate already mapped/scanned/catalogued (domain + data product exist) — `DGINSIGHT-Discovery.ps1`

• Least-privilege insights + data-quality roles assigned (verify current names) — `DGINSIGHT-Discovery.ps1`

• Data Quality enabled + its scan compute available (verify licensing/preview) — `DGINSIGHT-Discovery.ps1`

• Pilot data product with critical data elements identified — `DGINSIGHT-Set-Scope.ps1`

• Source connection + credentials via managed identity / Key Vault (no secrets in code)

• Quality dimensions, rules, and thresholds agreed with the owner — `DGINSIGHT-Set-Scope.ps1`

• Data-owner/steward review path + privacy sign-off for live-data scan

• Pilot scope and scan change window agreed

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires (E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Data Estate Insights & Data Quality (Microsoft Purview)

Follow these steps in order. This guide confirms the governed estate and roles, stands up **Data Estate Insights** (read-only reporting), authors **data-quality rules** on a bounded pilot data product, **pilots data-quality scoring on that bounded scope**, validates the scores and the health picture against the gate, then expands across the estate. Artifact names are fixed — create them with the exact names shown and reuse them everywhere. **Data-quality scoring is piloted on one bounded data product before the estate is scored.**

> *Model: read the estate's health, prove data-quality scoring on a pilot data product, then expand. The repeatable way: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.** Verify current portal paths, role names, capability availability, and API versions for your tenant as you go — the Purview insights and Data Quality experiences are evolving and parts may be in preview.*

Step 1 — Assess current state (read-only)

1. Run `DGINSIGHT-Discovery.ps1` to capture readiness signals: the Purview account endpoint is

reachable, how many **assets** are catalogued, which **governance domains** and **data products** exist, whether the **Data Quality** surface responds (feature available), and role coverage. No changes are made.

1. Record gaps against the Requirements checklist. Do **not** proceed past a missing governed estate,

missing least-privilege roles, or Data Quality not being enabled — remediate those first (a missing Data Map/catalog is a hard stop; Data Quality unavailable means the scoring steps are deferred).

Step 2 — Design the insights and data-quality scope

1. Choose the pilot **governance domain** `DGI-Domain-Pilot` and pilot **data product**

`DGI-DataProduct-Pilot` (an existing curated data product from the estate).

1. With the data owner, list the **critical data elements** as `DGI-CDE-<name>` (for example

`DGI-CDE-CustomerID`, `DGI-CDE-Email`), and for each one agree the **quality dimension(s)**, the **rule** intent, and the **pass threshold** (Requirement 2.6). Thresholds are the owner's determination.

1. Assign least-privilege roles at the narrowest scope: **catalog / Data Governance reader** to

insights viewers, **governance domain owner** / **data steward** to the curators, and **data-quality steward** to whoever authors and runs rules. Reserve Data Governance Administrator for a small named group. (Verify current role names for your tenant.)

Step 3 — Record the pilot scope (read-only manifest)

1. Run `DGINSIGHT-Set-Scope.ps1` to record the pilot scope: the domain, the data product, its CDEs, and

per-CDE the dimension / rule intent / threshold. This produces the scope manifest you configure against (read-only; writes nothing to Purview).

1. Confirm the manifest matches what the owner agreed before you author any rule.

Step 4 — Stand up Data Estate Insights and take a baseline (read-only)

1. Open the estate **insights / analytics / Health management** experience in the Purview portal (verify

the current path — classic "Data Estate Insights" vs the new analytics/health surface). Confirm you can see estate-level reports: asset counts, **classification distribution**, **ownership coverage**, scan health, glossary/data-product coverage.

1. Run `DGINSIGHT-Report.ps1` to capture the same picture as a **baseline** named `DGI-Insights-Baseline`

(asset totals, classified %, owned %, per-domain/data-product coverage). Insights are read-only — this step changes nothing and is safe to repeat.

1. Record the baseline. It is the "before" number the estate's health is tracked against as you expand.

Step 5 — Establish the data-quality connection and credentials (safe by design)

1. Configure the **data-quality connection** from the pilot data product to its backing source, using

the Purview account's **managed identity** or an **Azure Key Vault**–stored secret. **Do not put any secret in a script or document.**

1. Grant the identity the **least read** role on the source (the scan reads live data — least-privilege

is mandatory). Run **Test connection** and confirm it passes before authoring rules.

Step 6 — Author data-quality rules on the pilot data product

1. In `DGI-DataProduct-Pilot`, create the data-quality rule set `DGI-DQ-Ruleset-Pilot`. For each CDE,

author the agreed rule named `DGI-DQ-Rule-<Dimension>-<CDE>` (for example `DGI-DQ-Rule-Completeness-CustomerID`, `DGI-DQ-Rule-Conformity-Email`, `DGI-DQ-Rule-Uniqueness-CustomerID`) with the owner's **threshold**.

1. Use `DGINSIGHT-Configure-Quality.ps1` to stage the rules — it **defaults to** `-WhatIf` and prints

exactly the rules/thresholds it would create; re-run with `-Commit` to author them. Or author them in the portal.

1. Keep the scope to the **pilot data product only** — do not attach rules across the domain yet.

Step 7 — Pilot the data-quality scan on the bounded scope (test-first)

1. Trigger the **data-quality scan/assessment once** on `DGI-DataProduct-Pilot` only (not scheduled

yet). Use `DGINSIGHT-Configure-Quality.ps1 -RunScan` (`-WhatIf` by default; `-Commit` to run) or the portal. The scan **reads live data** on the bounded scope with the owner's agreement. Pass/fail: the scan completes **Succeeded** and returns a **quality score** per rule and a rolled-up score for the data product.

1. **Rollback:** a data-quality scan does **not modify source data** — it reads and scores. If it

over-reaches, runs long, or costs too much, cancel the run in the portal and set the rule set to not scheduled; re-scope to fewer CDEs/assets and re-run. To back out a mis-authored rule, disable or delete `DGI-DQ-Rule-*` (re-run `DGINSIGHT-Configure-Quality.ps1 -Remove -WhatIf` first to preview).

Step 8 — Review the scores with the owner and tune rules

1. Run `DGINSIGHT-Report.ps1` to list the per-rule and per-data-product **quality scores** and which

CDEs fall below threshold.

1. With the data owner, confirm each score reflects reality. Where a rule is wrong (false low/high),

tune the rule or threshold in `DGI-DQ-Ruleset-Pilot` and re-run the scan on the pilot scope. **The solution produces the score; the owner decides what the score means and whether remediation is required** — remediation of the underlying data is the owner's, not this solution's, action.

Step 9 — Wire estate-health tracking (health controls, if available)

1. If **Health management / health controls** is available in your tenant (verify — parts may be

preview), define a small health control set `DGI-Health-Controls-Pilot` for the pilot domain (for example: every data product has an owner; every CDE has at least one data-quality rule; no stale scan). Use `DGINSIGHT-Scope-Health.ps1` to report these coverage checks read-only.

1. If the health-controls surface is not available, record it as deferred and use

`DGINSIGHT-Scope-Health.ps1` alone to report the same coverage gaps — it does not depend on the feature being enabled.

Step 10 — Validate against the pilot gate

1. Complete the Testing Guide. The pilot passes only when: the insights baseline `DGI-Insights-Baseline`

is captured; the data-quality connection Test connection passes; `DGI-DQ-Ruleset-Pilot` is authored on `DGI-DataProduct-Pilot`; the data-quality scan **Succeeded** and returned a **quality score** per rule and a rolled-up score; and the owner has reviewed and accepted (or flagged remediation for) the scores.

1. Remediate any gaps and re-run the relevant validation before proceeding. Do **not** widen scope or schedule scoring until the gate passes.

Step 11 — Expand across the estate (enforce)

1. After the pilot passes, extend data-quality rules to further data products one domain at a time,

re-running `DGINSIGHT-Set-Scope.ps1`, `DGINSIGHT-Configure-Quality.ps1` (`-WhatIf` first), and `DGINSIGHT-Report.ps1` for each new scope.

1. Move data-quality scans from one-time to **scheduled** only once a scope is validated, so scores stay current as data changes.

1. Track estate health over time by re-running `DGINSIGHT-Report.ps1` against the `DGI-Insights-Baseline`

and widening `DGI-Health-Controls-Pilot` as coverage grows. Keep threshold and remediation decisions with the data owners.

Order of operations

```
Assess -> Design (domain/data product/CDEs/rules/thresholds) -> Record scope -> Insights baseline (read-only) -> DQ connection & credentials -> Author DQ rules (pilot data product) -> Pilot DQ scan (bounded, one-time) -> Review scores & tune -> Health tracking -> Validate gate -> Expand (scheduled scoring)
```

5. Testing Guide — Data Estate Insights & Data Quality (Microsoft Purview)

Run these tests after the insights baseline and the pilot data-quality scan (Implementation Steps 4–9) and again after each expansion scope. Each test confirms one part of the measure-and-report capability works end to end. All read-only checks are safe to repeat.

Test 1 — Estate and roles are ready

Do: Run `DGINSIGHT-Discovery.ps1`. Expected: The Purview account endpoint is reachable, catalogued **assets**, **governance domains**, and **data products** are reported, the **Data Quality** surface responds (feature available), and least-privilege roles are reported as assigned; no required role is missing. (If Data Quality does not respond, its tests below are recorded as deferred — verify licensing/preview — not failed.)

Test 2 — Data Estate Insights baseline is captured

Do: Open the insights/analytics/Health management experience and run `DGINSIGHT-Report.ps1`. Expected: An estate-level picture is produced — asset totals, **classification distribution**, **ownership coverage %**, and per-domain/data-product coverage — and captured as `DGI-Insights-Baseline`. The portal view and the report agree.

Test 3 — The data-quality connection is reachable

Do: Run **Test connection** on the pilot data-quality connection and confirm with `DGINSIGHT-Discovery.ps1`. Expected: The connection to the pilot data product's source passes with least-privilege read (managed identity or Key Vault credential); no secret appears in any script or manifest.

Test 4 — Data-quality rules are authored on the pilot data product

Do: Open `DGI-DataProduct-Pilot` and confirm `DGI-DQ-Ruleset-Pilot`; run `DGINSIGHT-Report.ps1`. Expected: The agreed `DGI-DQ-Rule-<Dimension>-<CDE>` rules exist against the pilot CDEs with the owner's thresholds; the rule set is scoped to the pilot data product only.

Test 5 — The data-quality scan succeeds and returns scores

Do: Review the pilot data-quality scan run in the portal and run `DGINSIGHT-Report.ps1`. Expected: The scan status is **Succeeded**, it scored the intended bounded scope, and a **quality score** is returned per rule, per data asset, and rolled up for `DGI-DataProduct-Pilot`. No source data was modified.

Test 6 — Scores are meaningful and owner-reviewed

Do: Run `DGINSIGHT-Report.ps1` and review the per-rule scores and below-threshold CDEs with the data owner. Expected: Each score reflects reality (false low/high rules tuned and re-run); the owner has accepted the scores or flagged which CDEs need remediation. The count of below-threshold rules is understood and owned — remediation is the owner's action, not a test failure.

Test 7 — Estate-health coverage is reported

Do: Run `DGINSIGHT-Scope-Health.ps1` (and, if available, review `DGI-Health-Controls-Pilot`). Expected: Coverage gaps are reported for the pilot domain — data products without an owner, CDEs without a data-quality rule, stale scans. Gaps are listed, not silently missing; where health-controls is unavailable the script still reports the same coverage.

Test 8 — Trend tracking works against the baseline

Do: Re-run `DGINSIGHT-Report.ps1` after a change (a new rule, a re-scan, or an expansion scope) and compare to `DGI-Insights-Baseline`. Expected: The report shows the delta from baseline (classification/ownership coverage and data-product quality score moving as expected). If Data Quality is unavailable in the tenant, this test covers the insights delta only and the DQ portion is recorded as deferred.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–6 pass: estate/roles ready, insights baseline captured, DQ connection reachable, rules authored on the pilot data product, the DQ scan **Succeeded** with per-rule and data-product scores, and the owner has reviewed/accepted the scores (Test 7 reported; Test 8 established). Where Data Quality is unavailable for the tenant, Tests 3–6 are recorded as **deferred** with the insights tests (1, 2, 7, 8-insights) passing.
Expand pass	Tests 1–7 pass for each new scope, and scheduled data-quality scoring is enabled only after the scope validates and the owner accepts its scores.

6. Troubleshooting Guide — Data Estate Insights & Data Quality (Microsoft Purview)

Use `DGINSIGHT-Discovery.ps1` for readiness/role/feature state, `DGINSIGHT-Report.ps1` for the insights baseline and data-quality scores, `DGINSIGHT-Set-Scope.ps1` to (re)record a scope, `DGINSIGHT-Scope-Health.ps1` for coverage gaps, and `DGINSIGHT-Configure-Quality.ps1` (`-WhatIf` by default) to author rules and run scoring. All scripts are read-only except `DGINSIGHT-Configure-Quality.ps1`, which changes nothing unless you pass `-Commit`.

A. Cannot reach the account / token fails

Symptom	Cause	Remediation
`Discovery` reports endpoint GAP	Wrong account name or endpoint	Confirm the account name; the endpoint is `https://<account>.purview.azure.com`; re-run `DGINSIGHT-Discovery.ps1`
Token acquisition fails	Not signed in / wrong resource	Run `Connect-AzAccount`; the scripts request a token for `https://purview.azure.net` — confirm your account can obtain it
401/403 on every call	Missing least-privilege role	Assign a catalog / Data Governance reader (view) or the specific role the action needs at the domain scope; re-run
Calls fail with "api-version" errors	The example API version has moved	The insights / Unified Catalog / Data Quality experience is evolving — verify and set the current data-plane API version for your tenant

B. Data Quality surface does not respond (feature availability)

Symptom	Cause	Remediation
`Discovery` reports Data Quality GAP	Feature not licensed/enabled or in preview	Confirm the required licensing/consumption model and enablement with your Purview/Azure owner; if unavailable in your tenant/region, record the DQ steps as **deferred** — do not assume availability
Data Quality reachable but 403	Missing the data-quality steward role	Assign the **data-quality steward** role on the governance domain (verify current role name); re-run
DQ menu absent in the portal	Portal path/name has moved	Verify the current Unified Catalog Data Quality path for your tenant — the experience is evolving

C. Insights show no data

Symptom	Cause	Remediation
`Report` returns 0 assets	Estate not scanned/catalogued yet	This solution measures a governed estate — stand up the Data Map / scanning / catalog first, then re-baseline
Classification distribution empty	Scans applied no classifications	Confirm scans ran with classifications in the Data Map solution; this solution reports what scanning produced, it does not classify
Ownership coverage 0%	No owners assigned on assets/data products	Assign owners in the Unified Catalog; unowned assets are a governance gap surfaced by `DGINSIGHT-Scope-Health.ps1`
Portal insights view differs from `Report`	Different scope or cache	Align the search scope; the report samples the catalog — for the authoritative view use the portal insights/analytics surface

D. The data-quality connection or Test connection fails

Symptom	Cause	Remediation
Test connection fails (cloud source)	Managed identity lacks read on the source	Grant the Purview account's managed identity the **least read** role on the source; re-test

Symptom	Cause	Remediation
"Credential not found"	Key Vault credential name wrong / vault not linked	Link the Key Vault to the account and reference the credential **by name**; never inline a secret in a script
Test connection fails (on-prem/private)	No network path	Use the appropriate integration runtime with line of sight to the source; confirm it is Running; re-test
Scan cannot read the CDE columns	Identity has table but not column access, or wrong asset mapped	Confirm the data product's assets map to the real source columns and the identity can read them; re-map and re-test

E. The data-quality scan fails or returns no scores

Symptom	Cause	Remediation
Scan status Failed	Transient / auth / compute unavailable	Open the run in the portal for the failure reason; confirm the connection and that Data Quality compute is available; re-run
Scan Succeeded, 0 rule scores	Rules not authored / not attached to the data product	Confirm `DGI-DQ-Ruleset-Pilot` exists on `DGI-DataProduct-Pilot` (`DGINSIGHT-Report.ps1`); author with `DGINSIGHT-Configure-Quality.ps1 -Commit`; re-scan
Scan runs long / costs too much	Scope too broad (whole source, all columns)	Re-scope to the bounded pilot CDEs; keep it one-time until validated; schedule only after the gate passes
`-Commit` seems to do nothing	Ran in default `-WhatIf`	The tool is safe by default — add `-Commit` (and `-RunScan -Commit` to start the scan)

F. Scores look wrong

- A false-low or false-high score usually means the **rule or threshold** is wrong, not the data.

Review with the **data owner** using `DGINSIGHT-Report.ps1`, tune the rule/threshold in `DGI-DQ-Ruleset-Pilot`, and re-scan the pilot scope. The solution produces the score; **the owner decides what it means and whether the data needs remediation** — a red score is evidence, not a compliance ruling, and remediating the underlying data is the owner's action.

G. Rolling back a rule or scan

- A data-quality scan **reads and scores; it does not modify source data**. To stop a run, cancel it in

the portal and set the rule set to not scheduled. To back out a mis-authored rule, preview with `DGINSIGHT-Configure-Quality.ps1 -Remove -WhatIf`, then apply with `-Remove -Commit`. Re-scope and re-run rather than widening scope.

H. Health / coverage gaps

Symptom	Cause	Remediation
`Scope-Health` shows owner GAP	Data product has no named owner	Assign an owner in the Unified Catalog; re-run `DGINSIGHT-Scope-Health.ps1`
CDEs-with-rule below total	Some CDEs have no data-quality rule	Author the missing `DGI-DQ-Rule-<Dimension>-<CDE>`; re-run the health check
"Never scored" / stale score	Scoring not run or beyond the window	Run/schedule scoring on the data product after the pilot validates; re-check with `-StaleDays`
Health-controls surface unavailable	Feature preview/not enabled	`DGINSIGHT-Scope-Health.ps1` reports the same coverage without it — record health-controls as deferred

I. Diagnostics quick reference

```
.\DGINSIGHT-Discovery.ps1 -PurviewAccount contoso-purview                                  # readiness: e
.\DGINSIGHT-Set-Scope.ps1 -Domain DGI-Domain-Pilot -DataProduct DGI-DataProduct-Pilot `
    -Cde @('DGI-CDE-CustomerID','DGI-CDE-Email')                                           # record the p
.\DGINSIGHT-Configure-Quality.ps1 -PurviewAccount contoso-purview -ScopeFile .\DGI-scope.json          # -
.\DGINSIGHT-Configure-Quality.ps1 -PurviewAccount contoso-purview -ScopeFile .\DGI-scope.json -RunScan -Co
.\DGINSIGHT-Report.ps1 -PurviewAccount contoso-purview -DataProduct DGI-DataProduct-Pilot   # insights bas
.\DGINSIGHT-Scope-Health.ps1 -PurviewAccount contoso-purview -Domain DGI-Domain-Pilot -StaleDays 30    # c
```

J. When to escalate

If the estate is reachable and catalogued, roles are assigned, Data Quality is confirmed available, the connection Test connection passes, and a scoped scan still returns no scores after confirming rules are authored and attached, capture the `DGINSIGHT-Discovery.ps1` and `DGINSIGHT-Report.ps1` output and the portal scan-run detail, and review the connection, the CDE-to-column mapping, and the rule set with the data owner and your Purview/Azure administrator. For evolving or preview features, verify current availability and API versions against Microsoft's documentation before assuming a defect.

7. Best Practices — Data Estate Insights & Data Quality (Microsoft Purview)

- Baseline before you tune. Capture `DGI-Insights-Baseline` first, then measure every change against

it. A quality or coverage number only means something relative to where you started — an unbaselined estate has no story to tell leadership.

• Pilot data-quality scoring on one data product. Data Quality reads live data and consumes compute;

prove the rules, the score, the runtime, and the cost on `DGI-DataProduct-Pilot` before you point scoring at the estate or schedule it. One-time first, scheduled only after a scope validates.

• Measure critical data elements, not everything. Rules bind to the columns that matter

(`DGI-CDE-<name>`). Scoring every column produces noise and cost; a small, agreed CDE set per data product produces a score the business will actually act on.

• Thresholds are the owner's call, not a technical default. Let the data owner set the pass threshold

per dimension (completeness ≥ 98%, etc.). The solution measures against the threshold; it does not decide what "good" is — that is a business determination.

• One rule, one dimension, one intent. Name rules `DGI-DQ-Rule-<Dimension>-<CDE>` so a score maps to a

clear question (is `Email` conformant? is `CustomerID` unique?). Compound rules produce scores nobody can act on.

• Never put a secret in a rule, scan, or script. Use the account's managed identity or an Azure Key

Vault–stored credential for the data-quality connection, granted least read on the source — this is both the secure pattern and what keeps the provided scripts secret-free.

• Treat a data-quality scan as reading real data. Unlike a classification scan, DQ profiling reads

actual values. Run it least-privilege, on a bounded scope, with the owner's privacy agreement — and record that it reads live data so nobody is surprised.

• The score is evidence, not a verdict. The solution produces a measured score; whether a low score is

a compliance issue and what remediation follows is the owner's/steward's determination. Keep that separation explicit so a red score is never mistaken for a compliance ruling.

• Wire health tracking so coverage doesn't rot. Use `DGINSIGHT-Scope-Health.ps1` (and health controls

where available) to catch data products without owners, CDEs without rules, and stale scans — a governed estate degrades quietly without a coverage check.

• Verify licensing, preview status, and portal paths as you go. Data Estate Insights / analytics /

Health management and Data Quality are evolving and consumption-/plan-gated, and parts may be in preview. Confirm current availability, naming, API versions, and pricing in Microsoft's docs and the portal rather than assuming a path, an endpoint, or a price is permanent.

8. Deliverables Summary — Data Estate Insights & Data Quality (Microsoft Purview)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box

Document	Purpose
1. Solution Overview	Data Estate Insights (estate health reporting) and Data Quality (rules + scoring across the governed estate), how a pilot works, and how it complements the Data Map/catalog foundation (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: design scope, baseline insights, author rules, pilot-scan scoring, review, health tracking, validate, expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

Named artifacts

Artifact	Name
Pilot governance domain	`DGI-Domain-Pilot`
Pilot data product	`DGI-DataProduct-Pilot`
Critical data element	`DGI-CDE-<name>`
Data-quality rule set	`DGI-DQ-Ruleset-Pilot`
Data-quality rule	`DGI-DQ-Rule-<Dimension>-<CDE>`
Insights baseline	`DGI-Insights-Baseline`
Health control set (if available)	`DGI-Health-Controls-Pilot`

PowerShell scripts

Script	What it does
`DGINSIGHT-Discovery.ps1`	Reports readiness: account endpoint reachable, catalogued asset / governance-domain / data-product counts, whether the Data Quality surface responds (feature available), and role coverage (Purview data-plane REST via an Azure token; read-only)
`DGINSIGHT-Set-Scope.ps1`	Records the pilot/expansion scope — domain, data product, critical data elements, and per-CDE dimension/rule/threshold — as a scope manifest to configure against (read-only)

Script	What it does
`DGINSIGHT-Configure-Quality.ps1`	Authors data-quality rules and runs the scoring scan for a data product; **defaults to** `-WhatIf` (prints the plan), applies only with `-Commit`; `-RunScan`/`-Remove` gated the same way; no secrets in code
`DGINSIGHT-Report.ps1`	Data Estate Insights + data-quality scoring report: asset/classification/ownership distribution and per-rule/per-data-product quality scores vs threshold, with a baseline for trend tracking (read-only)
`DGINSIGHT-Scope-Health.ps1`	Scope + estate-health helper: coverage gaps (data products without owners, CDEs without rules, stale scans) for a domain — reports the same checks whether or not health-controls is enabled (read-only)

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable method following the model: Assess → Design → Configure → Pilot → Validate → Expand.

Outcome

A measured, defensible data estate — health reported through Data Estate Insights (asset, classification, and ownership coverage) and data quality scored through Unified Catalog rules across the governed estate's critical data — piloted on a bounded data product and validated before scale, with thresholds and remediation left to the owners who make them.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DGINSIGHT-Configure-Quality.ps1

Purpose: Stages data-quality rules for a data product from a scope manifest and, with -RunScan, triggers the scoring scan. DEFAULTS TO -WhatIf: it prints exactly the rules/thresholds it would create (or the scan it would run) and changes NOTHING. Add -Commit to author rules; add -RunScan (with -Commit) to start the scoring scan; add -Remove to preview/delete a rule set. No credentials or secrets are stored — the data-quality connection uses the Purview managed identity / Key Vault configured in the portal. HOW TO RUN Preview the rules that would be authored (safe): powershell -ExecutionPolicy Bypass -File .\DGINSIGHT-Configure-Quality.ps1 ` -PurviewAccount contoso-purview -ScopeFile .\DGI-scope.json Author them for real: .\DGINSIGHT-Configure-Quality.ps1 -PurviewAccount contoso-purview -ScopeFile .\DGI-scope.json -Commit Run the scoring scan on the pilot data product: .\DGINSIGHT-Configure-Quality.ps1 -PurviewAccount contoso-purview -ScopeFile .\DGI-scope.json -RunScan -Commit Requires Az.Accounts + Connect-AzAccount as a Data Quality steward on the domain. EXPECTED RESULTS -WhatIf: a printed plan (rules + thresholds, or the scan to run) and NO changes. -Commit: each rule authored / scan started with a PASS line. A failure prints the reason and stops. .NOTES Purview — Data Quality. SAFE BY DEFAULT (-WhatIf). A data-quality scan READS LIVE DATA on the bounded scope — run least-privilege with the owner's agreement. API versions/paths below are EXAMPLES — verify current Data Quality API for your tenant (may be preview). No secrets in code. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [Parameter(Mandatory = $true)][string]$ScopeFile, [switch]$RunScan, [switch]$Remove, [switch]$Commit, [switch]$WhatIf = $true, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: Preview the rules that would be authored (safe): powershell -ExecutionPolicy Bypass -File .\DGINSIGHT-Configure-Quality.ps1 ` -PurviewAccount contoso-purview -ScopeFile .\DGI-scope.json Author them for real: .\DGINSIGHT-Configure-Quality.ps1 -PurviewAccount contoso-purview -ScopeFile .\DGI-scope.json -Commit Run the scoring scan on the pilot data product: .\DGINSIGHT-Configure-Quality.ps1 -PurviewAccount contoso-purview -ScopeFile .\DGI-scope.json -RunScan -Commit Requires Az.Accounts + Connect-AzAccount as a Data Quality steward on the domain. EXPECTED RESULTS -WhatIf: a printed plan (rules + thresholds, or the scan to run) and NO changes. -Commit: each rule authored / scan started with a PASS line. A failure prints the reason and stops. .NOTES Purview — Data Quality. SAFE BY DEFAULT (-WhatIf). A data-quality scan READS LIVE DATA on the bounded scope — run least-privilege with the owner's agreement. API versions/paths below are EXAMPLES — verify current Data Quality API for your tenant (may be preview). No secrets in code. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [Parameter(Mandatory = $true)][string]$ScopeFile, [switch]$RunScan, [switch]$Remove, [switch]$Commit, [switch]$WhatIf = $true, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

DGINSIGHT-Discovery.ps1

Purpose: Reports whether a Microsoft Purview estate is ready for Data Estate Insights and Data Quality: the account endpoint is reachable, how many assets are catalogued, which governance domains and data products exist, whether the Data Quality surface responds (feature available for the tenant), and least-privilege role coverage. Read-only — it queries the Purview data-plane REST and changes nothing. Use it in Implementation Step 1 to record gaps against the Requirements checklist. HOW TO RUN

powershell -ExecutionPolicy Bypass -File .\DGINSIGHT-Discovery.ps1 -PurviewAccount contoso-purview Requires the Az.Accounts module and Connect-AzAccount with at least a catalog/Data Governance reader role. No secrets are stored or requested. EXPECTED RESULTS A readiness summary: endpoint PASS/GAP, catalogued asset count, governance-domain and data-product counts, a Data-Quality-surface PASS/GAP (feature available?), and role coverage. GAP lines are what you remediate before authoring rules. .NOTES Purview — Data Estate Insights & Data Quality. Read-only. API versions below are EXAMPLES — verify current Purview data-plane / Unified Catalog / Data Quality API versions and availability for your tenant; the experience is evolving and parts may be in preview. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGINSIGHT-Discovery.ps1 -PurviewAccount contoso-purview Requires the Az.Accounts module and Connect-AzAccount with at least a catalog/Data Governance reader role. No secrets are stored or requested. EXPECTED RESULTS A readiness summary: endpoint PASS/GAP, catalogued asset count, governance-domain and data-product counts, a Data-Quality-surface PASS/GAP (feature available?), and role coverage. GAP lines are what you remediate before authoring rules. .NOTES Purview — Data Estate Insights & Data Quality. Read-only. API versions below are EXAMPLES — verify current Purview data-plane / Unified Catalog / Data Quality API versions and availability for your tenant; the experience is evolving and parts may be in preview. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

DGINSIGHT-Report.ps1

Purpose: Produces the estate's health-and-quality picture for a scope: from Data Estate Insights, the catalogued asset total, classification distribution, and ownership coverage %; from Data Quality, the per-rule and per-data-product quality scores vs threshold. Read-only — it queries the Purview data-plane REST and changes nothing. Use it to capture DGI-Insights-Baseline and to track the delta as the estate expands. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGINSIGHT-Report.ps1 -PurviewAccount contoso-purview ` -DataProduct DGI-DataProduct-Pilot Requires Az.Accounts + Connect-AzAccount with at least a catalog/Data Governance reader role. EXPECTED RESULTS An insights summary (assets, classified %, owned %) and, where Data Quality is available, a scores table (rule, dimension, score, threshold, PASS/BELOW). PASS/GAP flags map to the pass/fail gate. Where Data Quality is unavailable, the DQ section is reported as deferred, not failed. .NOTES Purview — Data Estate Insights & Data Quality. Read-only. API versions below are EXAMPLES — verify current Purview / Unified Catalog / Data Quality API versions for your tenant (parts may be preview). > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$DataProduct, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGINSIGHT-Report.ps1 -PurviewAccount contoso-purview ` -DataProduct DGI-DataProduct-Pilot Requires Az.Accounts + Connect-AzAccount with at least a catalog/Data Governance reader role. EXPECTED RESULTS An insights summary (assets, classified %, owned %) and, where Data Quality is available, a scores table (rule, dimension, score, threshold, PASS/BELOW). PASS/GAP flags map to the pass/fail gate. Where Data Quality is unavailable, the DQ section is reported as deferred, not failed. .NOTES Purview — Data Estate Insights & Data Quality. Read-only. API versions below are EXAMPLES — verify current Purview / Unified Catalog / Data Quality API versions for your tenant (parts may be preview). > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$DataProduct, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

DGINSIGHT-Scope-Health.ps1

Purpose: Reports estate-health coverage gaps for a governance domain, so a governed estate does not degrade quietly: data products without a named owner, critical data elements without a data-quality rule, and stale scans (last scored beyond a staleness window). Read-only — it queries the Purview data-plane REST and changes nothing. Reports the SAME coverage checks whether or not the tenant's Health management / health-controls surface is enabled, so it works even where that feature is preview/unavailable. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGINSIGHT-Scope-Health.ps1 ` -PurviewAccount contoso-purview -Domain DGI-Domain-Pilot -StaleDays 30 Requires Az.Accounts + Connect-AzAccount with at least a catalog/Data Governance reader role. EXPECTED RESULTS A coverage report per data product in the domain: owner PASS/GAP, CDEs-with-rules count, last-score age vs -StaleDays, and a rolled-up gap count you can record against DGI-Health-Controls-Pilot. .NOTES Purview — Data Estate Insights & Data Quality. Read-only. API versions below are EXAMPLES — verify current Unified Catalog / Data Quality API versions for your tenant (parts may be preview). > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [Parameter(Mandatory = $true)][string]$Domain, [int]$StaleDays = 30, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGINSIGHT-Scope-Health.ps1 ` -PurviewAccount contoso-purview -Domain DGI-Domain-Pilot -StaleDays 30 Requires Az.Accounts + Connect-AzAccount with at least a catalog/Data Governance reader role. EXPECTED RESULTS A coverage report per data product in the domain: owner PASS/GAP, CDEs-with-rules count, last-score age vs -StaleDays, and a rolled-up gap count you can record against DGI-Health-Controls-Pilot. .NOTES Purview — Data Estate Insights & Data Quality. Read-only. API versions below are EXAMPLES — verify current Unified Catalog / Data Quality API versions for your tenant (parts may be preview). > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [Parameter(Mandatory = $true)][string]$Domain, [int]$StaleDays = 30, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

DGINSIGHT-Set-Scope.ps1

Purpose: Captures the pilot scope as a local manifest you configure against: the governance domain, the data product, its critical data elements (CDEs), and per-CDE the quality dimension, rule intent, and pass threshold. Writes a JSON manifest to disk only — it changes NOTHING in Purview. Use it in Implementation Step 3 so rule authoring binds to exactly what the data owner agreed. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGINSIGHT-Set-Scope.ps1 `-Domain DGI-Domain-Pilot -DataProduct DGI-DataProduct-Pilot` -Cde @('DGI-CDE-CustomerID','DGI-CDE-Email') -OutFile .\DGI-scope.json No Purview connection and no credentials are required — this only records intent. EXPECTED RESULTS A DGI-scope.json manifest listing the domain, data product, and CDEs (with placeholders for dimension/rule/threshold to complete with the owner), plus a printed summary. Feed this file to DGINSIGHT-Configure-Quality.ps1 and DGINSIGHT-Report.ps1. .NOTES Purview — Data Estate Insights & Data Quality. Read-only (writes a local manifest only). Thresholds are a business determination set by the data owner — this tool records them, it does not decide them. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$Domain, [Parameter(Mandatory = $true)][string]$DataProduct, [Parameter(Mandatory = $true)][string[]]$Cde, [string]$OutFile = ".\DGI-scope.json")

How to run: powershell -ExecutionPolicy Bypass -File .\DGINSIGHT-Set-Scope.ps1 `-Domain DGI-Domain-Pilot -DataProduct DGI-DataProduct-Pilot` -Cde @('DGI-CDE-CustomerID','DGI-CDE-Email') -OutFile .\DGI-scope.json No Purview connection and no credentials are required — this only records intent. EXPECTED RESULTS A DGI-scope.json manifest

listing the domain, data product, and CDEs (with placeholders for dimension/rule/threshold to complete with the owner), plus a printed summary. Feed this file to DGINSIGHT-Configure-Quality.ps1 and DGINSIGHT-Report.ps1. .NOTES Purview — Data Estate Insights & Data Quality. Read-only (writes a local manifest only). Thresholds are a business determination set by the data owner — this tool records them, it does not decide them. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$Domain, [Parameter(Mandatory = $true)][string]$DataProduct, [Parameter(Mandatory = $true)][string[]]$Cde, [string]$OutFile = ".\DGI-scope.json")